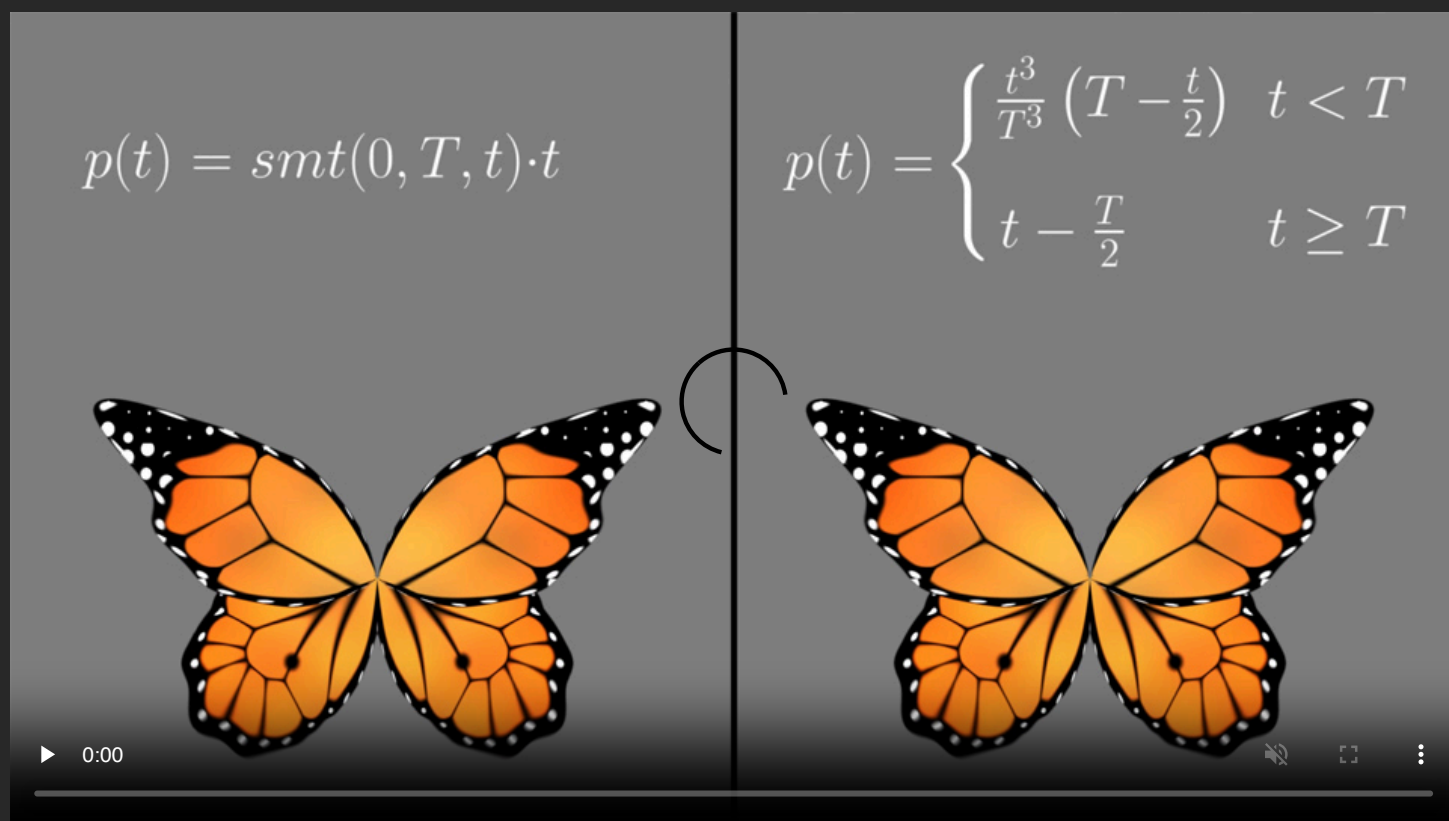


Inigo Quilez :: articles :: Smoothstep Integral - 2021



Intro

When doing stateless animations we somehow have to describe the final position and orientation of objects in a scene as a function of time, rather than as the result of a simulation process. That means that the concepts of velocity and acceleration are not given explicitly for a position to be computed, but they are rather implicit and a consequence of the position function that we design. And so, sometimes it can get a bit difficult to design a position function that produces a velocity that we desire.



Left, incorrect animation. Right, correct animation.

One such case is that of generating a smooth transition between a stationary situation to a motion of constant velocity. We do know that a `smoothstep()` function, also known sometimes as `EaseInOut()`, is a convenient way to smoothly interpolate values or transition between states. So it is natural to imagine that a `smoothstep()` might need to be involved in the design of such animation. However one has to be careful and smoothstep the right quantity, which

can be not obvious to some people. So, let's have a look:

The wrong way to do it

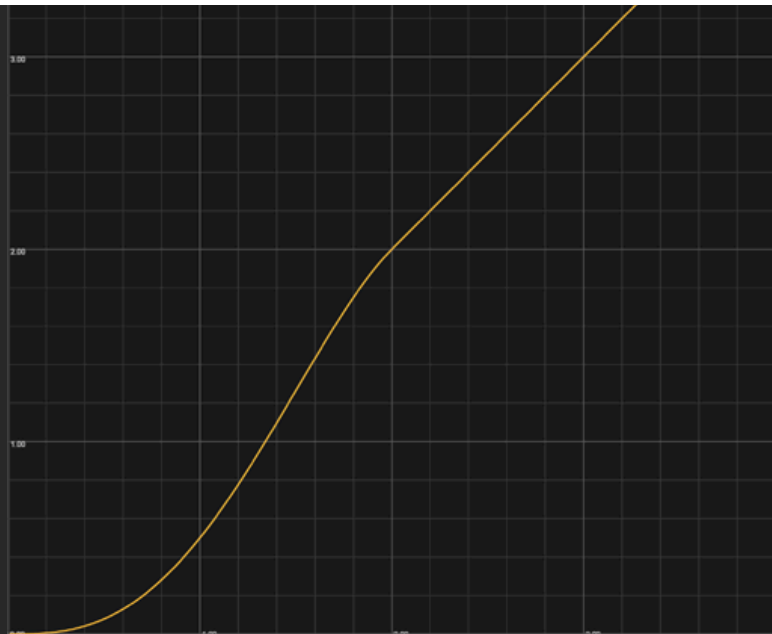
One approach to the design of our animation function is to notice that what we want is to smoothly interpolate between two velocities - from a speed of zero at the beginning of our animation to a constant velocity. Without loss of generality, let's assume we want to speed to a speed of 1 unit per second, and that we want this transition between stationary and constant velocity states to last T seconds. Basically

```
float velocity smoothstep(0.0,T,t);
```

And this is great so far actually. The problem is how to produce a position for our object based on this velocity. One might naively try multiplying velocity by time in order to get position, like this:

```
float position( float t, in float T )  
{  
    return smoothstep(0.0,T,t) * t;  
}
```

This however, does not work - when used in our scene, the object we are wanting to put in motion seems to speed up quickly and then very abruptly and unnaturally decelerate before settling in at constant speed. In the video at the top of the article, on the left side, you can see how the wings of the butterfly, which we are trying to get into motion, seem to slow down before starting to flap in a regular fashion. Certainly, it feels nothing like a smoothstep, and clearly our intuition has failed us here. Plotting the actual position() function over time actually reveals indeed that something went really wrong, and that indeed the slope of our position curve got temporarily steeper than 45 degrees, or 1, which was our target velocity:



The correct solution

So, what was the mistake, why didn't we get a smooth animation. Well, the problem is that we assumed that position is velocity multiplied by time, and that only works for constant velocities. In general, position is the integral of velocity. So in order to get our position function we need to integrate the smoothstep. Thankfully, since smoothstep is a simple cubic function, integrating it is very easy:

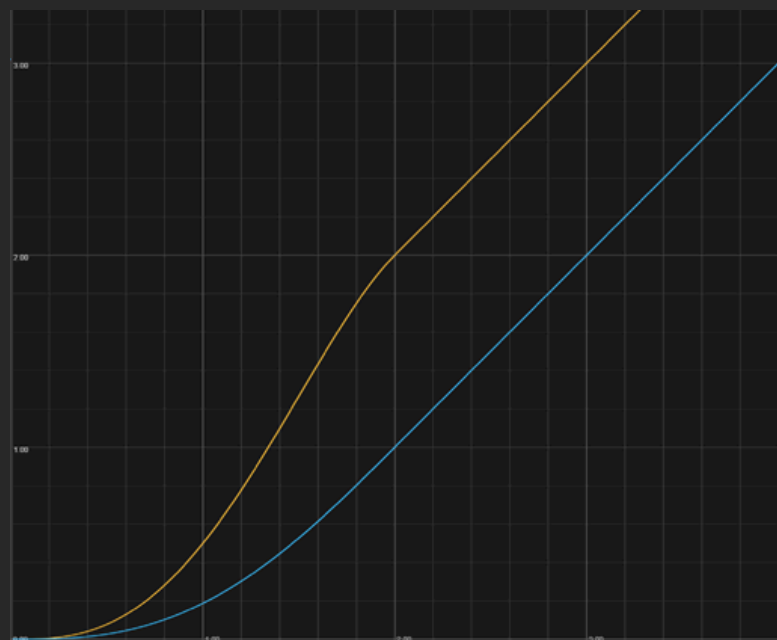
$$\begin{aligned}
 p(t) &= \int_0^t \text{smoothstep}(0, T, \lambda) d\lambda \\
 &= \int_0^t \begin{cases} \left(\frac{\lambda}{T}\right)^2 \left(3 - 2\frac{\lambda}{T}\right) d\lambda & t < T \\ d\lambda & t \geq T \end{cases} \\
 &= \begin{cases} \frac{t^3}{T^3} \left(T - \frac{t}{2}\right) & t < T \\ t - \frac{T}{2} & t \geq T \end{cases}
 \end{aligned}$$

This is just one division (if T is not known in advance) and a few multiplications and additions, and translates to code

directly as

```
float position( float t, in float T )
{
    if( t>=T ) return t - 0.5*T;
    float f = t/T;
    return f*f*f*(T-t*0.5);
}
```

The graph for this function, now in blue in the picture below, shows indeed a beautiful transition where the slope smoothly goes from zero to one without ever overshooting:



And indeed, when we use it for animation of the wings of the butterfly, shows in the right half of the video at the top of the article, the animation is smooth and just perfect.

You have a running example of the butterfly with the wrong and with the correct position function in Shadertoy:

<https://www.shadertoy.com/view/sdBSWc>. Also, I use this function often enough that I have added it to my collection of [Useful Little Functions](#). And lastly, I createad a [1 minute video summary](#) of this article.

inigo quilez - learning computer graphics since 1994